RELATORIO TECNICO

Conflitos de Dados e de Controle no Pipeline

Avaliacao Pratica M2 - Etapa 2 de 2
Organizacao de Computadores | UNIVALI

INTEGRANTES

Jose Gabriel Santos Gomes

Matheus Pompeo Dias

Nathan Gustavo Reichert

Objetivo

Estender o decodificador da Pratica M1 para que, a partir do mesmo arquivo de memoria de instrucoes, o programa detecte os conflitos de dados e de controle de um pipeline de cinco estagios sem nenhuma protecao de hardware e produza duas versoes corrigidas do codigo: uma somente com nops e outra com reordenacao de instrucoes e nops. Os desvios sao recalculados e os arquivos de saida podem ser lidos novamente pelo decodificador da M1.

Premissas do modelo

Pipeline IF, ID, EX, MEM, WB, sem adiantamento, deteccao de conflitos, descarte ou predicao. O banco de registradores e escrito na primeira metade do ciclo e lido na segunda, de modo que um consumidor a distancia **d** do produtor tem conflito quando d = 1 ou d = 2 e precisa de **3 - d** nops. Desvios sao resolvidos no ID e o slot de atraso sempre executa. O registrador zero nunca gera conflito, acessos com o mesmo registrador-base e deslocamentos diferentes sao enderecos distintos e **nop** e **addi zero, zero, 0** (0x00000013).

Ideia central: fluxo de execucao

O programa e tratado como uma sequencia de posicoes e a pergunta e quem executa logo depois de cada uma. Normalmente e a proxima. Depois de um desvio ou salto vem sempre o slot de atraso e so entao o fluxo se divide entre o alvo e a instrucao seguinte. Com essa relacao, a distancia entre produtor e consumidor e calculada por qualquer caminho, inclusive atravessando desvios, e a mesma funcao serve para detectar os conflitos do original e para verificar o resultado corrigido.

ENTRADA	ETAPAS	SAIDA
ROM em hex/bin (formato da M1)	Parser, Decoder (M1), analise de fluxo, nops, reordenacao, recalculo de desvios	Dois programas corrigidos (.hex/.bin), duas listagens (.lst) e relatorio de custo no console

1. Requisitos atendidos pelo projeto

A tabela relaciona cada requisito do enunciado com o que foi implementado e onde.

REQ.	O QUE PEDE	IMPLEMENTACAO NO PROJETO
R1 1,5	Detectar pares produtor/consumidor que violam a distancia minima, com enderecos, registrador e distancia. Somente leitura apos escrita; respeitar os campos de cada formato; produtor mais recente.	findHazards em src/hazards.cpp anda 1 e 2 posicoes para tras pelos predecessores no fluxo de execucao. Usa readsRegisters/writesRegister da M1, que deixam vazios os campos que o formato nao tem (S e B nao escrevem) e omitem x0.
R2 1,0	Identificar desvios e o ocupante do slot, formar blocos basicos e detectar conflitos de dados que atravessam o desvio.	makeContext cria os blocos (alvo inicia, desvio encerra). analyze lista desvio e slot. Conflitos cujo caminho sai de um slot sao marcados crossesBranch e impressos em secao propria.
R3 1,5	Menor numero de nops, sem mudar a ordem, sem conflito de dados restante.	insertDataNops corrige o primeiro consumidor em conflito, antes dele, e recalcula ate zerar. O alvo do desvio passa a apontar para o primeiro nop. Se o consumidor e o slot, os nops entram antes do desvio.
R4 1,0	Preencher o slot de atraso com nop.	buildItems insere um nop depois de cada desvio ou salto (Tag::NopSlot), pois o original supoe que a proxima instrucao nao executa.
R5 2,0	Reordenar dentro do bloco, preencher o slot com instrucao util e reverificar o inicio dos dois caminhos.	Scheduler: busca exata com memoizacao dentro do bloco (guloso acima de 14 instrucoes). slotCandidates e a escolha por bloco, testando cada candidata contra o programa inteiro.
R6 1,5	Recalcular deslocamentos de desvio e salto e reescrever em linguagem de maquina.	finalize calcula alvo novo menos posicao nova e withBranchOffset reempacota os bits embaralhados de B e J, com verificacao de alcance.
R7 1,0	Dois arquivos no formato da entrada, legiveis pelo decodificador da M1, e listagem com marcacoes.	writeOutputs em src/main_m2.cpp gera _nops e _reordenado (.hex ou .bin) e .lst com endereco, codigo, instrucao e marcacao.
R8 0,5	Custo de cada tecnica e comparacao.	printCost mostra original, final, acrescimo e percentual (virgula decimal) e compara as duas solucoes.

Distribuicao da nota no enunciado

CRITERIO	PONTOS	CRITERIO	PONTOS
R1 - dados	1,5	R5 - reordenacao	2,0
R2 - controle	1,0	R6 - enderecos	1,5
R3 - nops de dados	1,5	R7 - arquivos de saida	1,0
R4 - nops de controle	1,0	R8 - relatorio de custo	0,5
		Total	10,0

A defesa e individual e pode ajustar a nota de cada integrante.

2. Algoritmo

Estrutura de dados

Cada posicao do programa e um **Item**: a instrucao decodificada, o indice no original (ou -1 para nop inserido), o bloco basico, o bloco de destino quando e desvio e uma etiqueta (original, reordenada, movida para o slot, nop de dados, nop de slot). O mesmo tipo representa a entrada e as saidas. O alvo de um desvio e guardado como bloco, nao como endereco, e so vira numero na etapa final.

Deteccao (R1 e R2)

Para cada instrucao e cada registrador que ela le, procuram-se os produtores em 1 e 2 passos para tras. Se a instrucao imediatamente anterior no caminho escreve o registrador, a distancia e 1. Caso contrario, olha-se a anterior a ela e a distancia e 2. O produtor mais recente encerra a busca naquele caminho. Escrita apos leitura e escrita apos escrita nao entram.

Nops de dados (R3)

Corrige-se o consumidor mais cedo no programa, inserindo antes dele o maior numero de nops que seus conflitos pedem, e a analise e refeita. Como os nops ficam antes do consumidor, valem para todos os caminhos que chegam a ele. O recalculo a cada insercao evita contar duas vezes nops que beneficiam varios consumidores.

Reordenacao (R5)

Dentro de cada bloco, a dependencia entre duas instrucoes existe quando ha leitura apos escrita, escrita apos leitura, escrita apos escrita, ordem entre acessos a memoria que podem se sobrepor, ou uma delas e barreira (ecall, CSR, palavra invalida). Acessos com o mesmo registrador-base, deslocamentos diferentes e base inalterada entre eles sao independentes. O desvio depende de todas as instrucoes do bloco.

A ordem e escolhida por busca exata: o estado e o conjunto de instrucoes ja emitidas mais os registradores escritos pelas duas ultimas. O custo de emitir uma instrucao e 2 nops se ela le o que a ultima escreve e 1 se le o que a penultima escreve. Empates ficam com a ordem original.

Uma instrucao pode ocupar o slot de atraso se vem de antes do desvio e nenhuma outra instrucao do bloco, nem o proprio desvio, depende dela. Preencher o slot costuma criar um conflito novo no inicio dos caminhos, entao cada candidata e avaliada contra o programa inteiro, ja com os nops de dados, e o slot com nop so e mantido se for estritamente melhor. A escolha e repetida bloco a bloco ate estabilizar.

Recalculo de enderecos (R6)

Com a sequencia final fixada, o deslocamento de cada desvio e salto e: endereco novo do alvo menos endereco novo da instrucao. Os bits sao reempacotados nas posicoes embaralhadas do formato B (imm[12|10:5|4:1|11]) ou J (imm[20|10:1|11|19:12]) e a palavra e decodificada de novo. Um desvio para logo depois da ultima instrucao acompanha o novo fim do programa. Deslocamento fora do alcance (13 bits em B, 21 bits em J) e erro.

3. Validacao e resultados

Verificacoes automaticas (tests/test_hazards.cpp)

VERIFICACAO	COMO FUNCIONA	RESULTADO
Equivalencia de comportamento	Emulador RV32I com duas maquinas: ideal (sem conflito nem slot) e pipeline do enunciado (leitura defasada, slot). O estado final do original na maquina ideal deve ser igual ao de cada saida no pipeline.	1500 programas aleatorios com lacos, desvios para frente e jal: todos iguais. 1306 deles mudam de resultado sem a correcao.
Sem conflitos restantes	A deteccao e executada de novo sobre o resultado.	0 em todos os casos
Otimalidade	Busca exaustiva independente de todas as permutacoes validas em trechos retos curtos.	400 trechos: tamanho sempre igual ao minimo
Codificacao de desvios	Reescreve deslocamentos e decodifica de volta com o decodificador da M1.	Todos os valores e registradores preservados
Mutacao	Desligar a regra de escrita apos leitura na reordenacao.	O teste falha (comportamento muda)

Casos de teste (tests/data)

CASO	O QUE COBRE	ORIGINAL	SO NOPS	REORDENADO
cadeia	Cadeia de dependencias sem instrucao independente	5	13	13
sem_nops	Reordenacao elimina todos os nops	9	15	9
laco	Laco com desvio para tras	8	13	9
frente	Desvio para frente	7	14	14
enunciado	Laco no estilo do exemplo do enunciado	9	17	14

Em cadeia, cada consumidor usa o resultado da instrucao anterior, nao ha instrucao independente para preencher o espaco e a reordenacao nao ajuda: o resultado e igual ao de somente nops (4 consumidores com 2 nops cada). Em sem_nops, intercalar os tres pares produtor/consumidor deixa todos a distancia 3 e nenhum nop e necessario.

Exemplo de saida no console

```
CUSTO
  original ..........   9 instrucoes
  somente nops ......   17 instrucoes  (+8, +88,9%)
  nops + reordenacao    14 instrucoes  (+5, +55,6%)

Comparacao: a reordenacao evitou 3 de 8 nops (37,5%).
```

Limitacoes

jalr nao tem alvo conhecido: o slot e tratado, o destino nao. auipc e relativo ao PC e muda quando ha insercao de nops (o programa avisa). Acessos de tamanhos diferentes que se sobrepoem com a mesma base e deslocamentos diferentes seguem a premissa 6 do enunciado. jal e jalr tem slot de atraso como os desvios condicionais.

Como executar

```
cmake -S . -B build && cmake --build build
build/riscv-hazards programa.hex -b 0x00400000 -o saida
build/test_hazards
```